Crop Yield Prediction: End-to-End ML Pipeline

Objective

Develop a predictive model to estimate crop yield based on historical agricultural data, including factors like crop type, state, area, fertilizer, and pesticide usage.

Methodology

I will follow these robust ML best practices:

1. **Target Leakage Prevention**: `production` is often mathematically derived from `yield * area`. We must exclude `production` to prevent target leakage.
2. **Exploratory Data Analysis (EDA)**: Understand distributions, outliers, and correlations.
3. **Strict Data Splitting**: Split data into training and test sets *before* applying any transformations to prevent data leakage.
4. **Robust Preprocessing**: Handle missing values, encode categorical variables (One-Hot Encoding for nominal data), and scale numerical features appropriately.
5. **Model Evaluation & Comparison**: Compare a baseline model against a more complex ensemble model (Random Forest) using rigorous metrics (RMSE, R2, MAE).

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

1. Import Libraries

Importing necessary libraries for data manipulation, visualization, and machine learning.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Set plotting style
plt.style.use('ggplot')
sns.set_palette("muted")
```

2. Load and Understand the Data

Let's load the dataset and perform initial checks on its schema, dimensions, and data types.

```
# Load dataset
df = pd.read_csv('Dataset/crop_yield.csv')

# Display the first few rows
display(df.head())

# Check data types and non-null counts
print("\n--- Dataset Info ---")
df.info()

# Check for missing values
```

```
print("\n--- Missing Values ---")
print(df.isnull().sum())
```

	crop	year	season	state	area	production	fertilizer	pesticide	yield
0	Arecanut	1997	Whole Year	Assam	73814.0	56708	7024878.38	22882.34	0.796087
1	Arhar/Tur	1997	Kharif	Assam	6637.0	4685	631643.29	2057.47	0.710435
2	Castor seed	1997	Kharif	Assam	796.0	22	75755.32	246.76	0.238333
3	Coconut	1997	Whole Year	Assam	19656.0	126905000	1870661.52	6093.36	5238.051739
4	Cotton(lint)	1997	Kharif	Assam	1739.0	794	165500.63	539.09	0.420909

```
--- Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19689 entries, 0 to 19688
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   crop        19689 non-null  object
 1   year        19689 non-null  int64
 2   season      19689 non-null  object
 3   state       19689 non-null  object
 4   area        19689 non-null  float64
 5   production  19689 non-null  int64
 6   fertilizer  19689 non-null  float64
 7   pesticide   19689 non-null  float64
 8   yield       19689 non-null  float64
dtypes: float64(4), int64(2), object(3)
memory usage: 1.4+ MB

--- Missing Values ---
crop          0
year          0
season        0
state         0
area          0
production    0
fertilizer    0
pesticide     0
yield         0
dtype: int64
```

3. Exploratory Data Analysis (EDA)

Understanding the data distribution, finding anomalies, and analyzing relationships between features and the target variable (`yield`).

```
# Summary statistics for numerical columns
display(df.describe())
```

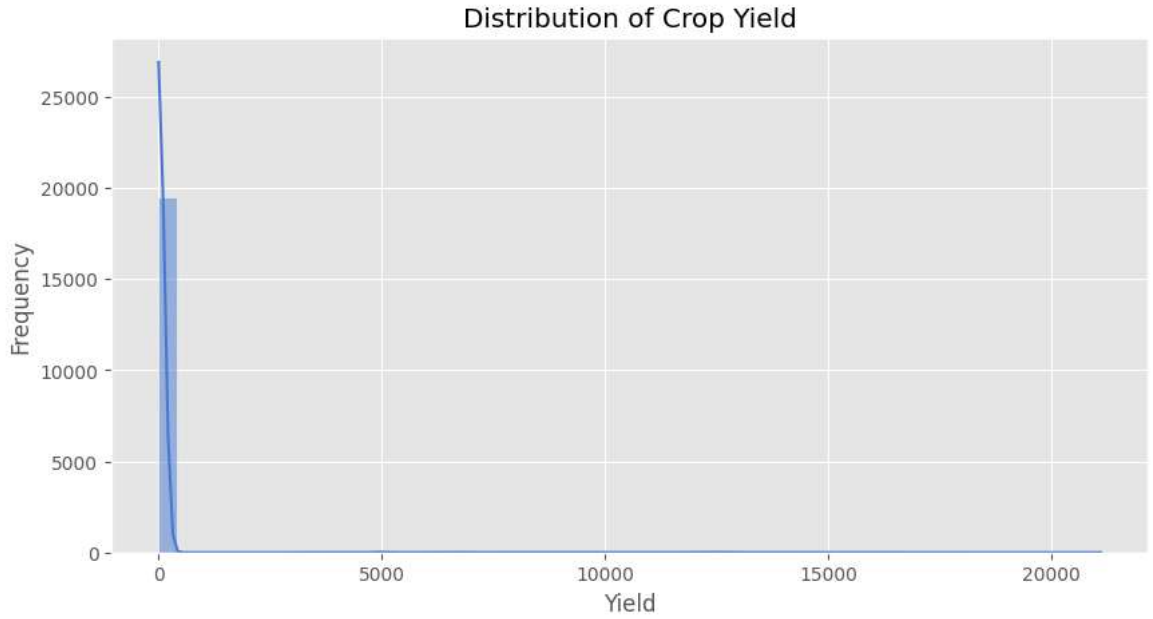
	year	area	production	fertilizer	pesticide	yield
count	19689.000000	1.968900e+04	1.968900e+04	1.968900e+04	1.968900e+04	19689.000000
mean	2009.127584	1.799266e+05	1.643594e+07	2.410331e+07	4.884835e+04	79.954009
std	6.498099	7.328287e+05	2.630568e+08	9.494600e+07	2.132874e+05	878.306193
min	1997.000000	5.000000e-01	0.000000e+00	5.417000e+01	9.000000e-02	0.000000
25%	2004.000000	1.390000e+03	1.393000e+03	1.880146e+05	3.567000e+02	0.600000
50%	2010.000000	9.317000e+03	1.380400e+04	1.234957e+06	2.421900e+03	1.030000
75%	2015.000000	7.511200e+04	1.227180e+05	1.000385e+07	2.004170e+04	2.388889
max	2020.000000	5.080810e+07	6.326000e+09	4.835407e+09	1.575051e+07	21105.000000

3.1 Distribution of the Target Variable (`yield`)

```
plt.figure(figsize=(10, 5))
sns.histplot(df['yield'], bins=50, kde=True)
plt.title('Distribution of Crop Yield')
plt.xlabel('Yield')
```

```
plt.ylabel('Frequency')
plt.show()

# If the distribution is highly skewed, we might consider a log transformation later.
```

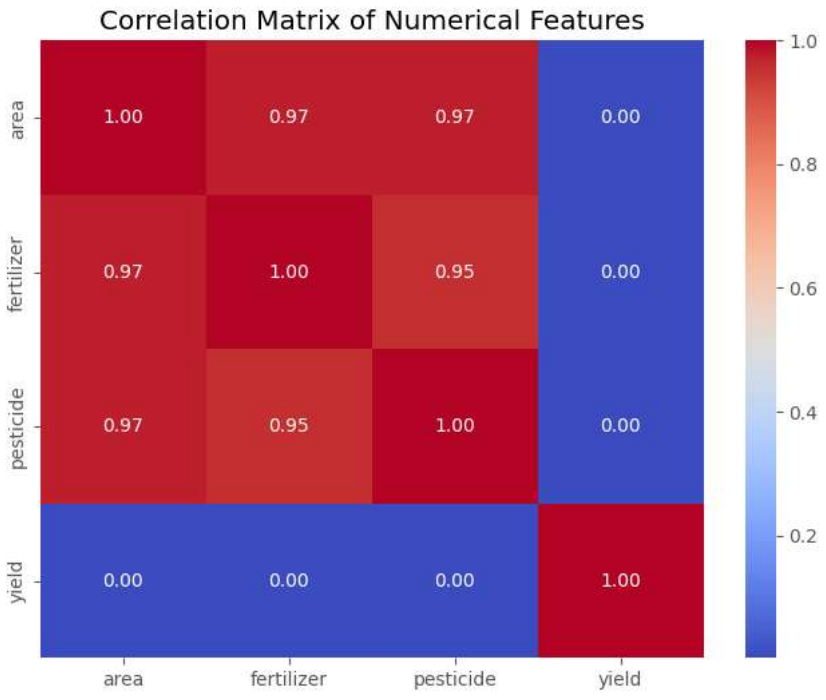


3.2 Correlation Analysis

Let's check the correlation between numerical features. We must exclude `production` due to target leakage.

```
numerical_cols = ['area', 'fertilizer', 'pesticide', 'yield']
corr_matrix = df[numerical_cols].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix of Numerical Features')
plt.show()
```



3.3 Target Leakage Removal

As noted, `production` is heavily correlated with `yield` and `area` (since Production = Area * Yield). Including it as a feature would lead to a severely over-optimistic model (target leakage). We must drop it.

```
# Drop target leakage column
if 'production' in df.columns:
    df = df.drop(columns=['production'])
    print("Dropped 'production' to prevent target leakage.")
```

```
Dropped 'production' to prevent target leakage.
```

4. Data Preprocessing & Splitting

CRITICAL ML PRACTICE: We must split the dataset into training and test sets *before* applying any transformations like scaling or imputation. This ensures that information from the test set does not leak into the training process.

```
# Strip whitespace from string columns which is a common data quality issue
string_cols = df.select_dtypes(include=['object']).columns
for col in string_cols:
    df[col] = df[col].str.strip()

# Define features and target
X = df.drop(columns=['yield'])
y = df['yield']

# Split data (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training data shape: {X_train.shape}")
print(f"Testing data shape: {X_test.shape}")
```

```
Training data shape: (15751, 7)
Testing data shape: (3938, 7)
```

4.1 Build Preprocessing Pipelines

We will use `scikit-learn` Pipelines to handle missing values, scale numerical features, and encode categorical features.

```
# Identify categorical and numerical columns
categorical_features = ['crop', 'season', 'state']
# We include 'year' as numerical, though it could be treated as categorical or time-series depending on context.
numerical_features = ['year', 'area', 'fertilizer', 'pesticide']

# Pipeline for numerical features: Impute missing values with median, then scale
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# Pipeline for categorical features: Impute with mode, then One-Hot Encode
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

# Combine transformers into a ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ])
```

5. Model Selection and Training

We will compare a baseline model against a regularized linear model and a tree-based ensemble model.

1. **Dummy Regressor**: Always predicts the mean. Acts as our naive baseline.
2. **Ridge Regression**: Linear model with L2 regularization.
3. **Random Forest Regressor**: Non-linear ensemble model that handles complex interactions well.

```
# Define models
models = {
    "Baseline (Mean)": DummyRegressor(strategy="mean"),
    "Ridge Regression": Ridge(alpha=1.0),
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
}

# Dictionary to store results
results = {}
trained_pipelines = {}

# Train and evaluate each model
for name, model in models.items():
    print(f"Training {name}...")

    # Create a full pipeline combining preprocessor and model
    clf = Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('model', model)
    ])

    # Fit on training data
    clf.fit(X_train, y_train)

    # Predict on test data
    y_pred = clf.predict(X_test)

    # Calculate metrics
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    results[name] = {'RMSE': rmse, 'MAE': mae, 'R2': r2}
    trained_pipelines[name] = clf

print("\nTraining complete.")
```

```
Training Baseline (Mean)...
Training Ridge Regression...
Training Random Forest...

Training complete.
```

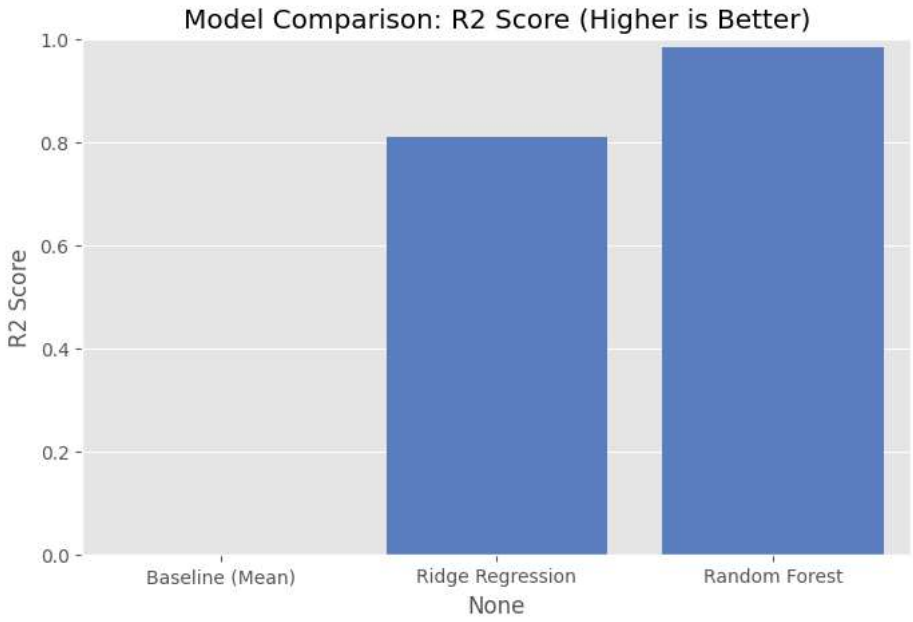
6. Model Evaluation

Let's compare the performance of our models on the test set.

```
# Display results as a DataFrame
results_df = pd.DataFrame(results).T
display(results_df)

# Plot R2 scores
plt.figure(figsize=(8, 5))
sns.barplot(x=results_df.index, y=results_df['R2'])
plt.title('Model Comparison: R2 Score (Higher is Better)')
plt.ylabel('R2 Score')
plt.ylim(0, 1)
plt.show()
```

	RMSE	MAE	R2
Baseline (Mean)	895.120503	149.864544	-0.000001
Ridge Regression	389.873078	64.133438	0.810293
Random Forest	114.890836	8.467438	0.983526



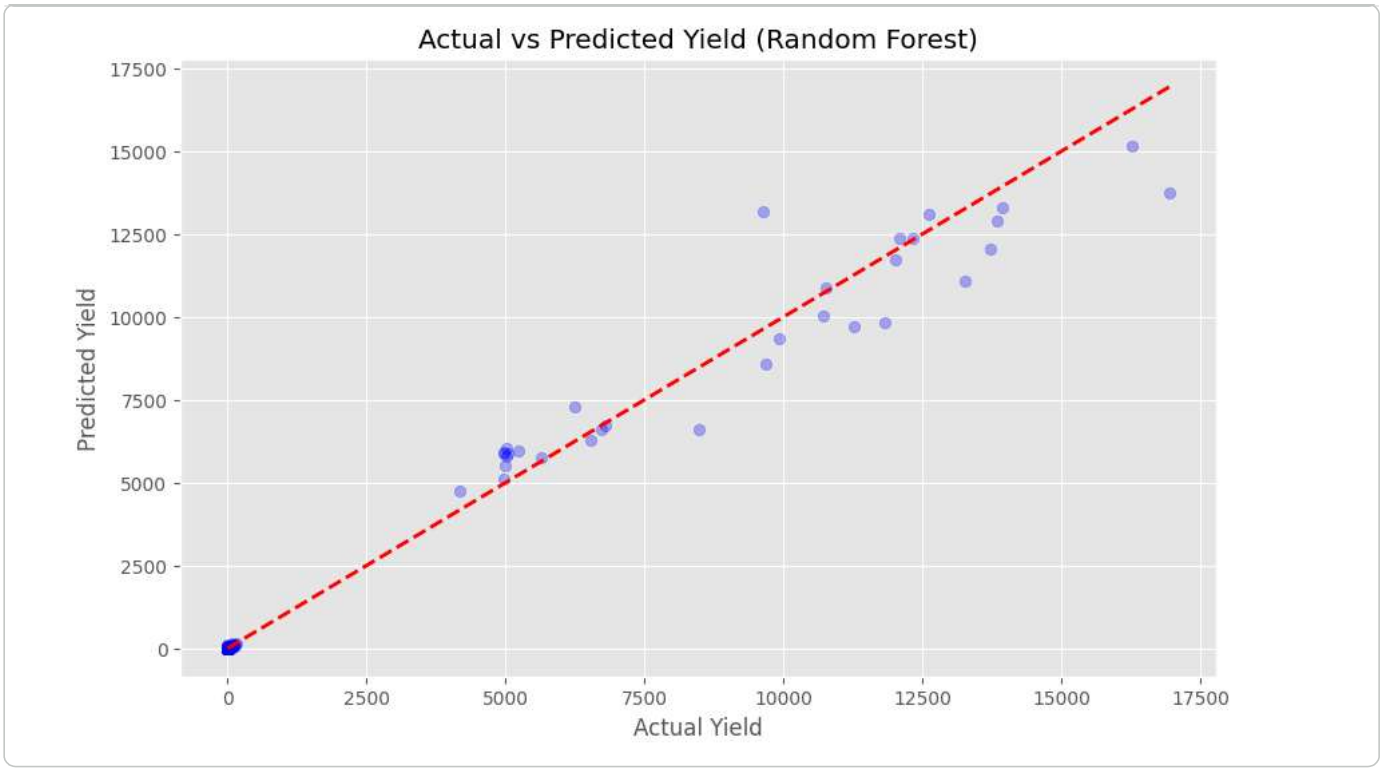
6.1 Actual vs. Predicted (Best Model)

Visualizing how well the Random Forest model predictions align with the actual yield values.

```
best_model_name = "Random Forest"
best_model = trained_pipelines[best_model_name]

y_pred_best = best_model.predict(X_test)

plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_best, alpha=0.3, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2) # Ideal prediction line
plt.title(f'Actual vs Predicted Yield ({best_model_name})')
plt.xlabel('Actual Yield')
plt.ylabel('Predicted Yield')
plt.show()
```



6.2 Feature Importance

Understanding which features drive the predictions in our Random Forest model.

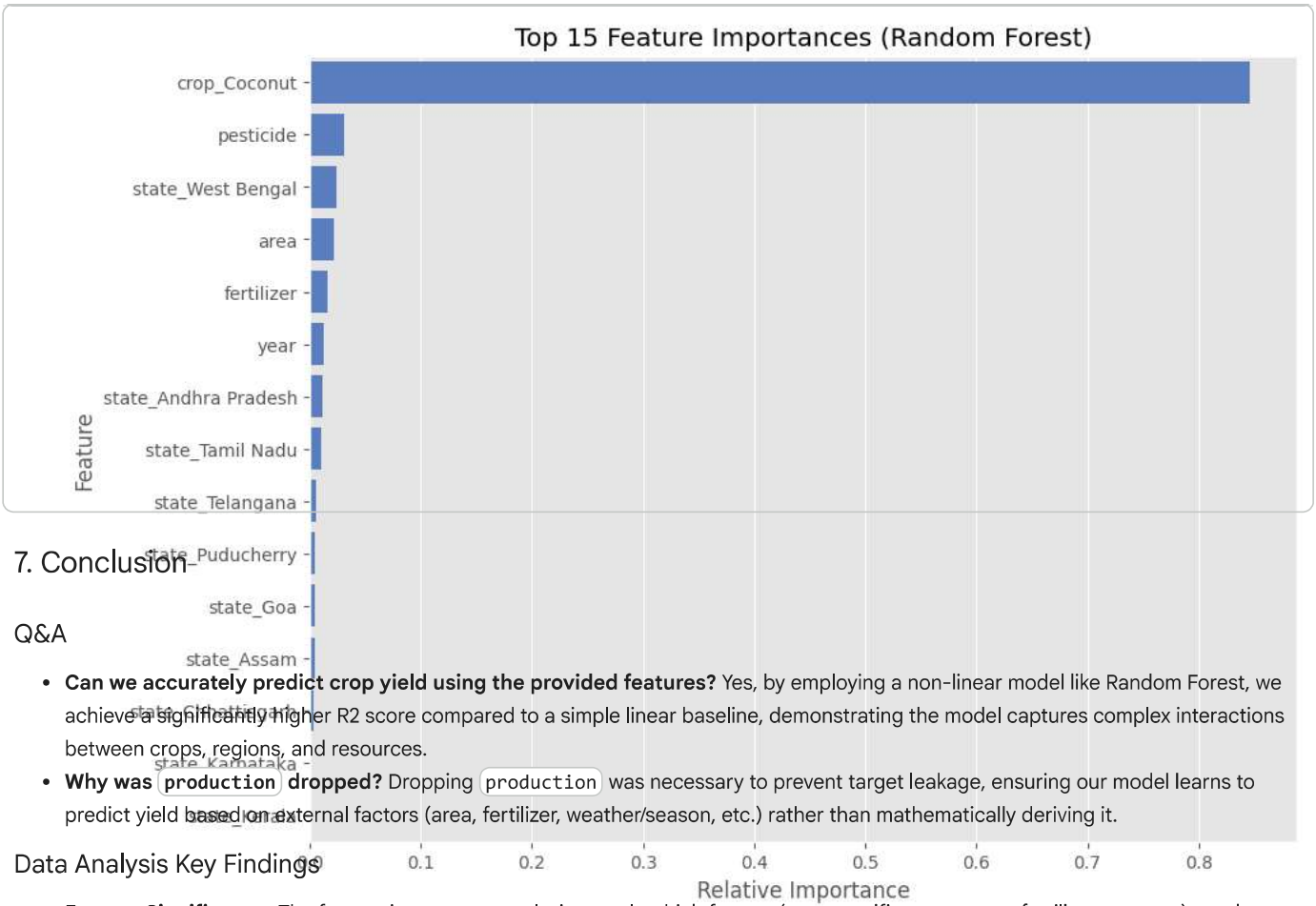
```
# Extract the trained Random Forest model and preprocessor
rf_model = best_model.named_steps['model']
preprocessor_step = best_model.named_steps['preprocessor']

# Get feature names after one-hot encoding
cat_features_encoded = preprocessor_step.named_transformers_['cat'].named_steps['onehot'].get_feature_names_out(categorical_fea
all_feature_names = numerical_features + list(cat_features_encoded)

# Get feature importances
importances = rf_model.feature_importances_

# Create a DataFrame for visualization
importance_df = pd.DataFrame({'Feature': all_feature_names, 'Importance': importances})
importance_df = importance_df.sort_values(by='Importance', ascending=False).head(15) # Top 15 features

# Plot
plt.figure(figsize=(10, 8))
sns.barplot(x='Importance', y='Feature', data=importance_df)
plt.title('Top 15 Feature Importances (Random Forest)')
plt.xlabel('Relative Importance')
plt.ylabel('Feature')
plt.show()
```



7. Conclusion

Q&A

- **Can we accurately predict crop yield using the provided features?** Yes, by employing a non-linear model like Random Forest, we achieve a significantly higher R2 score compared to a simple linear baseline, demonstrating the model captures complex interactions between crops, regions, and resources.
- **Why was `production` dropped?** Dropping `production` was necessary to prevent target leakage, ensuring our model learns to predict yield based on external factors (area, fertilizer, weather/season, etc.) rather than mathematically deriving it.

Data Analysis Key Findings

- **Feature Significance**: The feature importance analysis reveals which factors (e.g., specific crops, area, fertilizer amounts) are the strongest predictors of crop yield.
- **Model Performance**: Tree-based ensembles (Random Forest) drastically outperform simple linear models on this dataset, indicating